Trabajo Práctico 4 – Planificación

SISTEMAS OPERATIVOS

Índice

1. El planificador y las políticas de planificación **3**

2. Cambios de contexto **4**

3. Observar procesos con `top` **5**

4. Prioridad con `nice` y `renice` **6**

5. Experimento de planificación **6**

6. Control de trabajos **7**

Este TP estudia cómo Linux decide qué tarea obtiene la CPU (Central Processing Unit) y durante cuánto tiempo. Vas a observar prioridades, políticas de planificación, cambios de contexto y desalojos usando comandos y archivos del sistema. Los valores concretos dependen de la carga, la cantidad de núcleos y la versión del núcleo: registrá tus observaciones en lugar de copiar una salida de ejemplo.

1. El planificador y las políticas de planificación

El planificador (*scheduler*) elige entre las tareas que están listas para ejecutarse. Una tarea puede ceder la CPU voluntariamente al bloquearse, por ejemplo al esperar entrada y salida (E/S, *input/output*), o puede ser desalojada cuando el núcleo necesita ejecutar otra tarea.

POSIX (*Portable Operating System Interface*) define una interfaz para consultar y modificar políticas de planificación. Linux implementa varias de ellas; en este TP nos concentraremos en estas tres políticas y en la llamada que permite ceder la CPU:

- `SCHED_OTHER`: es la política normal de tiempo compartido. Linux intenta repartir la CPU entre las tareas aptas y calcula una prioridad dinámica a partir de su comportamiento y de su valor de `nice`. No es conveniente modelarla como una planificación por turnos (*Round Robin*) simple con un quantum (intervalo de tiempo, *time slice*) fijo.
- `SCHED_FIFO` (*First In, First Out*): pertenece a las políticas de tiempo real y usa una prioridad estática. Una tarea continúa ejecutándose hasta bloquearse, ceder la CPU, terminar o ser desalojada por otra tarea de tiempo real con prioridad mayor. Las tareas con la misma prioridad no reciben un quantum automático.
- `SCHED_RR` (*Round Robin*): es similar a `SCHED_FIFO`, pero las tareas de tiempo real con la misma prioridad se alternan mediante un intervalo de tiempo.
- `sched_yield()`: no es una política adicional. Es una llamada que expresa que la tarea actual está dispuesta a ceder la CPU; su efecto depende de la política y de las demás tareas listas.

Las políticas de tiempo real pueden afectar a todo el sistema. En general, un usuario común no puede asignárselas a sus procesos sin permisos especiales. No las pruebes sobre procesos del sistema ni ejecutes una tarea de tiempo real que nunca bloquee o termine en una máquina compartida.

```
$ man 7 sched
$ ps -eo pid,ppid,ni,pri,rtprio,stat,psr,comm --sort=pid | head
$ chrt -p 1234
```

En la salida de `ps`, `NI` es el valor de `nice`, `PRI` es la prioridad que muestra `procps` y `RTPRIO` es la prioridad de tiempo real cuando corresponde. `PSR` indica el procesador lógico en el que se observó la tarea. La salida exacta y

la interpretación de algunas columnas dependen de la política y de la versión de las herramientas.

EXTRA

Si tenés disponible el código fuente del núcleo de tu distribución, hacé esta actividad; no hace falta leer todo el núcleo:

1. Consultá la política del shell actual con `chrt -p $$` y anotá la salida.
2. Ubicá la definición de `__schedule()` con `grep -R -n __schedule kernel/sched/`.
3. Leé esa función e identificá dónde se obtiene la siguiente tarea (por ejemplo, mediante `pick_next_task`) y dónde se realiza el cambio de contexto (por ejemplo, mediante `context_switch`).
4. Entregá una tabla de dos filas y tres columnas: concepto descrito por `man 7 sched`, archivo y símbolo encontrados, y relación entre ambos. Las filas deben cubrir la selección de la siguiente tarea y el cambio de contexto. Si tu versión usa otros nombres o rutas, registrá la diferencia.

2. Cambios de contexto

Un cambio de contexto (*context switch*) ocurre cuando el núcleo deja de ejecutar una tarea y guarda su estado para poder continuar luego con otra. El sistema de archivos virtual `/proc` (*proc filesystem*) expone parte de esta información como si fueran archivos.

El contador global de cambios de contexto aparece en la línea `ctxt` de `/proc/stat`:

```
$ grep '^ctxt ' /proc/stat
ctxt 123456789
```

Ese valor es acumulado desde el arranque y puede aumentar mientras observás la salida. Repetí el comando varias veces: incluso una máquina que parece inactiva ejecuta tareas de fondo, interrupciones y servicios.

Para estudiar un proceso particular, mirá su archivo `status`. Las dos métricas siguientes separan las cesiones voluntarias de los desalojos involuntarios:

```
$ pid=$$
$ grep -E 'voluntary_ctxt_switches|nonvoluntary_ctxt_switches' \
    /proc/$pid/status
voluntary_ctxt_switches: 4
nonvoluntary_ctxt_switches: 0
```

El contador de cesiones voluntarias aumenta cuando el proceso se bloquea o cede la CPU de forma explícita. El contador de desalojos involuntarios

aumenta cuando el núcleo lo desaloja, por ejemplo para dar paso a otra tarea lista. No esperes que uno de los dos sea siempre cero: un mismo programa puede hacer ambas cosas.

La variable especial `$$` contiene el PID (*Process ID*) del shell actual. En cambio, `/proc/self/status` se refiere al proceso que abre el archivo. Si ejecutás `grep ... /proc/self/status`, normalmente estarás observando al propio `grep`, que vive muy poco tiempo. Para observar el shell u otro proceso, guardá su PID y construí `/proc/$pid/status` como en el ejemplo.

El desalojo no es sinónimo de hiperpaginación (*thrashing*). La hiperpaginación es un problema de memoria en el que el sistema pasa demasiado tiempo moviendo páginas entre la memoria principal y el área de intercambio. Muchos cambios de contexto pueden tener un costo de planificación, pero deben analizarse junto con la carga de CPU y de memoria.

EXTRA

Si está instalado, compará el contador de `/proc` con `pidstat -w 1` o `vmstat 1`. ¿Qué información es global y cuál corresponde a un proceso? También podés explorar `/proc/interrupts`, sin modificar ningún archivo del sistema.

3. Observar procesos con `top`

El comando `top` actualiza continuamente una lista de tareas y permite cambiar su orden y enviarles señales. Probalo en una terminal:

```
$ top
$ top -H
```

La opción `-H` muestra hilos (*threads*) en lugar de agruparlos por proceso. Dentro de `top`, las teclas más útiles para este TP son:

- `P`: ordenar por porcentaje de CPU; `M`: ordenar por memoria;
- `1`: mostrar el uso de cada CPU lógica; `H`: alternar la vista de hilos;
- `k`: enviar una señal al PID elegido; `q`: salir.

Las columnas habituales son:

- `PID`, `USER` y `COMMAND`: identificador, propietario y comando de la tarea;
- `PR` y `NI`: prioridad mostrada y valor de `nice`. En procesos normales, un valor de `NI` más alto representa menor preferencia relativa; los valores negativos requieren permisos adicionales;
- `VIRT`, `RES` y `SHR`: memoria virtual total, memoria residente y parte residente potencialmente compartida;
- `S` o `STAT`: estado. `R` es ejecutando o listo, `S` es sueño interrumpible, `D` es sueño ininterrumpible, `T` es detenido o trazado y `Z` es zombie;

- `%CPU`, `%MEM` y `TIME+`: uso reciente de CPU, proporción de memoria física y tiempo de CPU acumulado.

El orden de la lista puede cambiar aun cuando no escribas ningún comando. El planificador actualiza prioridades dinámicas, aparecen y terminan tareas, y en una máquina con varios procesadores distintas tareas pueden ejecutarse en paralelo. Por eso una instantánea de `top` no alcanza para demostrar una regla universal.

4. Prioridad con `nice` y `renice`

El valor de `nice` modifica la preferencia relativa de los procesos de tiempo compartido. En Linux el rango usual va de -20 a 19: un valor menor representa más prioridad relativa y un valor mayor representa menos. El comando `nice` crea el proceso con un incremento de niceness; el comando `renice` modifica un proceso que ya existe.

```
$ nice -n 10 comando
$ renice +5 -p 1234
```

En el primer ejemplo, `10` hace que el proceso sea menos preferido que uno con `NI=0`. En el segundo, `+5` incrementa el valor de niceness del PID indicado y también reduce su prioridad relativa. Bajar el valor de niceness, por ejemplo a -5, normalmente requiere permisos administrativos. No confundas `nice` con una medición de velocidad: si la CPU está libre, dos procesos con valores distintos pueden usar casi toda la CPU disponible.

5. Experimento de planificación

Vamos a comparar dos trabajos que consumen CPU. `yes` imprime una secuencia interminable de letras; al redirigir su salida a `/dev/null` la descartamos y dejamos visible principalmente el trabajo del procesador. Abrí `top` en una terminal y usá otra para ejecutar lo siguiente:

```
$ yes > /dev/null &
[1] 829
$ pid1=$!
$ nice -n 10 yes > /dev/null &
[2] 830
$ pid2=$!
```

Los números del ejemplo cambian en cada sistema. `&` inicia el comando en segundo plano (*background*), `[1]` es el número de trabajo del intérprete de comandos (*shell*) y el otro número es su PID. `$!` guarda el PID del último proceso iniciado en segundo plano. Observá ambos procesos en `top` durante varios segundos y luego consultá sus métricas:

```
$ ps -o pid,ppid,ni,pri,rtprio,stat,pcpu,comm -p "$pid1,$pid2"
$ grep -E 'voluntary_ctxt_switches|nonvoluntary_ctxt_switches' \
    /proc/$pid1/status
$ grep -E 'voluntary_ctxt_switches|nonvoluntary_ctxt_switches' \
    /proc/$pid2/status
```

Ahora cambiá la prioridad relativa del primer proceso, siempre hacia una prioridad menor, y volvé a observar:

```
$ renice +5 -p "$pid1"
$ ps -o pid,ni,pri,pcpu,stat,comm -p "$pid1,$pid2"
```

Registrá una tabla con los valores de `NI`, `PR`, `%CPU` y los dos contadores de contexto antes y después de `renice`. Contestá:

1. ¿Qué cambió de manera inmediata y qué cambió sólo después de esperar?
2. ¿El proceso con mayor niceness siempre recibió menos CPU? Repetí la prueba con poca carga y luego con dos o más procesos competidores.
3. ¿Qué efecto tiene la cantidad de núcleos? Relacioná la respuesta con `PSR`, con la vista por CPU de `top` y con el hecho de que dos procesos pueden ejecutarse en paralelo.
4. Compará los cambios de contexto voluntarios y no voluntarios. Proponé una explicación para cualquier diferencia, separando lo que observaste de lo que esperabas observar.

Al terminar, finalizá los procesos usando una señal normal y esperá a que el shell recoja sus estados:

```
$ kill "$pid1" "$pid2"
$ wait "$pid1" "$pid2" 2>/dev/null
```

INFO

`kill` envía una señal: no significa necesariamente “matar”. La señal predeterminada es `SIGTERM`, que permite a un programa terminar ordenadamente. `Ctrl-C` envía normalmente `SIGINT` al trabajo en foreground. Reservá `SIGKILL` (`kill -9`) para un proceso que no responde y verificá siempre el PID antes de usarlo.

6. Control de trabajos

Como ya viste en el TP2, el intérprete de comandos distingue entre el primer plano (*foreground*), que ocupa la terminal, y el segundo plano (*background*), que permite volver al prompt. El número de trabajo sólo tiene sentido dentro de ese shell; el PID identifica al proceso en el sistema. Practicá el ciclo completo con un comando que tarde lo suficiente:

```
$ sleep 100
```

Mientras está en foreground, apretá `Ctrl-Z`. Luego ejecutá:

```
$ jobs
$ bg %1
$ jobs
$ fg %1
```

El número `1` es sólo un ejemplo: usá el que muestre tu shell. Con `bg` el trabajo continúa en background y con `fg` vuelve al foreground. Podés detenerlo otra vez con `Ctrl-Z` y terminarlo con `kill %1`. Relacioná cada acción con los estados que veas en `jobs`, `ps` y `top`.

EXTRA

Explorá `taskset -p PID` para observar la afinidad de una tarea y repetí el experimento usando una sola CPU, sólo si la máquina es tuya y sabés que no vas a afectar a otros usuarios. Como extensión, investigá `getrusage()` y la diferencia entre tiempo de usuario y tiempo de sistema.